MACHINE LEARNING

Module Code: CSC83101

LECTURE

TUYISHIMIRE Joseph

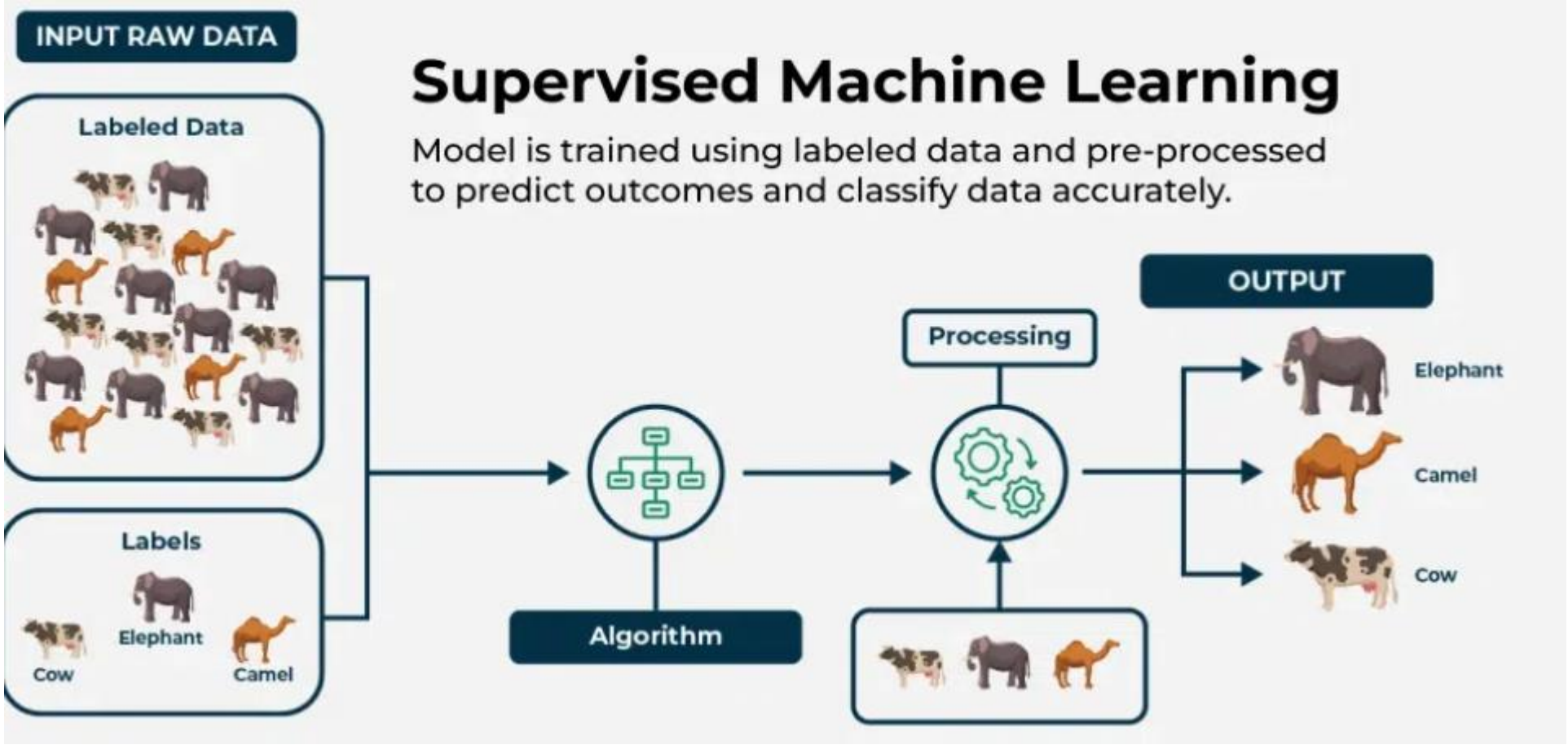
tuyishimirejo1992@gmail.com

UNIVERSITY Of KIGALI

Supervised Machine Learning

- Supervised learning is a type of machine learning where a model learns from labelled data meaning every input has a corresponding correct output.
- The model makes predictions and compares them with the true outputs, adjusting itself to reduce errors and improve accuracy over time. The goal is to make accurate predictions on new, unseen data. For example, a model trained on images of handwritten digits can recognise new digits it has never seen before.

UNIVERSITY Of KIGALI


INPUT RAW DATA
Supervised Machine Learning
Model is trained using labeled data and pre-processed to predict outcomes and classify data accurately.
Labeled Data
Labels
Cow
Elephant
Camel
Algorithm
Processing
OUTPUT
Elephant
Camel
Cow

Types of Supervised Learning in Machine Learning

- Now, Supervised learning can be applied to two main types of problems:
- **Classification**: Where the output is a categorical variable (e.g., spam vs. non-spam emails, yes vs. no).
- **Regression**: Where the output is a continuous variable (e.g., predicting house prices, stock prices)

While training the model, data is usually split in the ratio of 80:20 i.e. 80% as training data and the rest as testing data. In training data, we feed input as well as output for 80% of data. The model learns from training data only.

We use different supervised learning algorithms (which we will discuss in detail in the next section) to build our model. Let's first understand the classification and regression data through the table below

Working of Supervised Machine Learning

The working of supervised machine learning follows these key steps:

1. **Collect Labeled Data**

Gather a dataset where each input has a known correct output (label).

Example: Images of handwritten digits with their actual numbers as labels.

2. Split the Dataset

Divide the data into training data (about 80%) and testing data (about 20%).

The model will learn from the training data and be evaluated on the testing data.

3. Train the Model

Feed the training data (inputs and their labels) to a suitable supervised learning algorithm (like Decision Trees, SVM or Linear Regression).

The model tries to find patterns that map inputs to correct outputs

UNIVERSITY Of KIGALI

4. Validate and Test the Model

Evaluate the model using testing data it has never seen before.

The model predicts outputs and these predictions are compared with the actual labels to calculate accuracy or error.

5. Deploy and Predict on New Data

Once the model performs well, it can be used to predict outputs for completely new, unseen data.

Supervised Machine Learning Algorithms

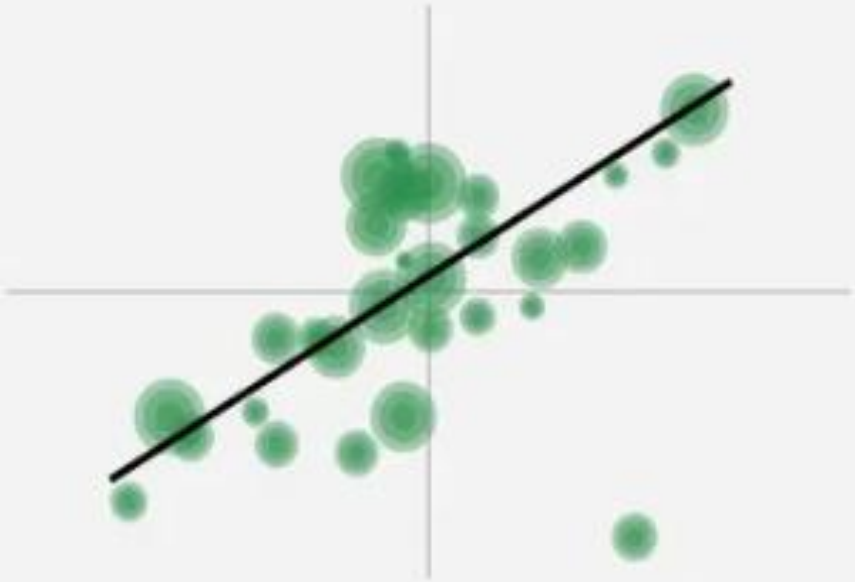
Supervised learning can be further divided into several different types, each with its own unique characteristics and applications. Here are some of the most common types of supervised learning algorithms:

Linear Regression in Machine learning

Linear regression is a type of supervised machine-learning algorithm that learns from the labelled datasets and maps the data points with most optimized linear functions which can be used for prediction on new datasets. It assumes that there is a linear relationship between the input and output, meaning the output changes at a constant rate as the input changes. This relationship is represented by a straight line

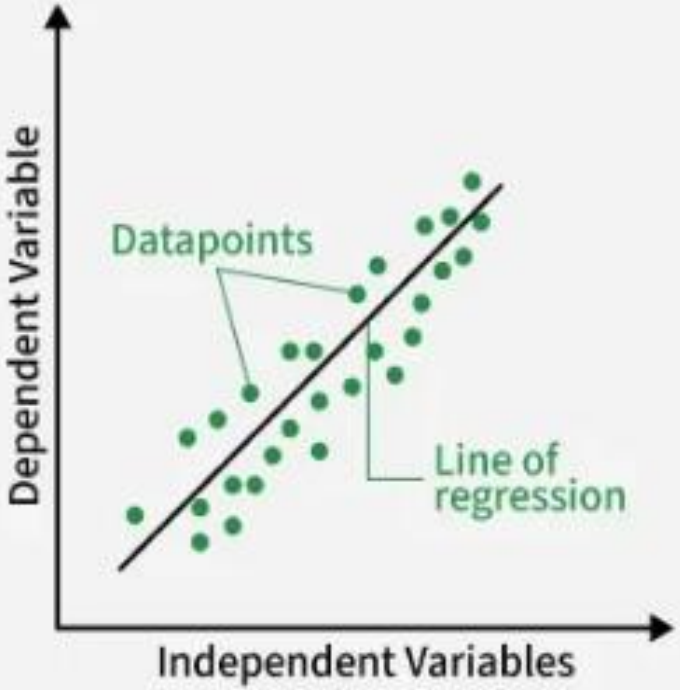
Introduction to Linear Regression

- Simple model for predicting continuous values.
- It finds the relationship between input features and output.
- It is used in forecasting and predicting salaries based on experience.



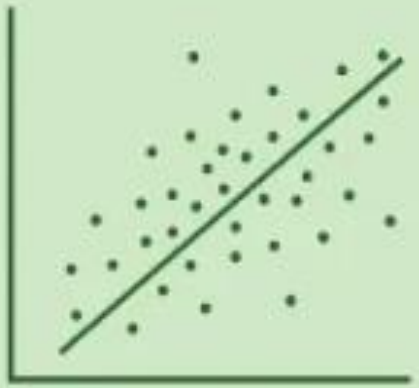
How Does Linear Regression Work?

- Finds the best-fitting line by minimizing prediction errors (least squares method)
- It calculates coefficients for variables that minimize the error in predictions.



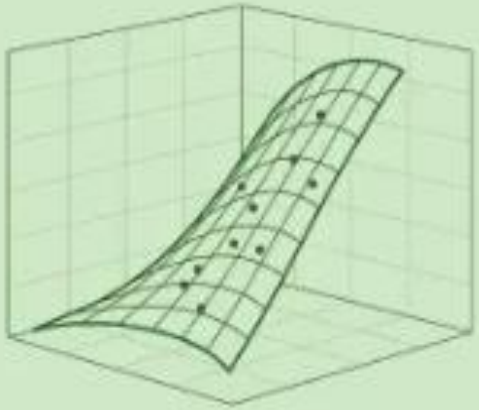
Types of Linear Regression

Simple Linear Regression



Predicts the dependent variable using a single independent variable.

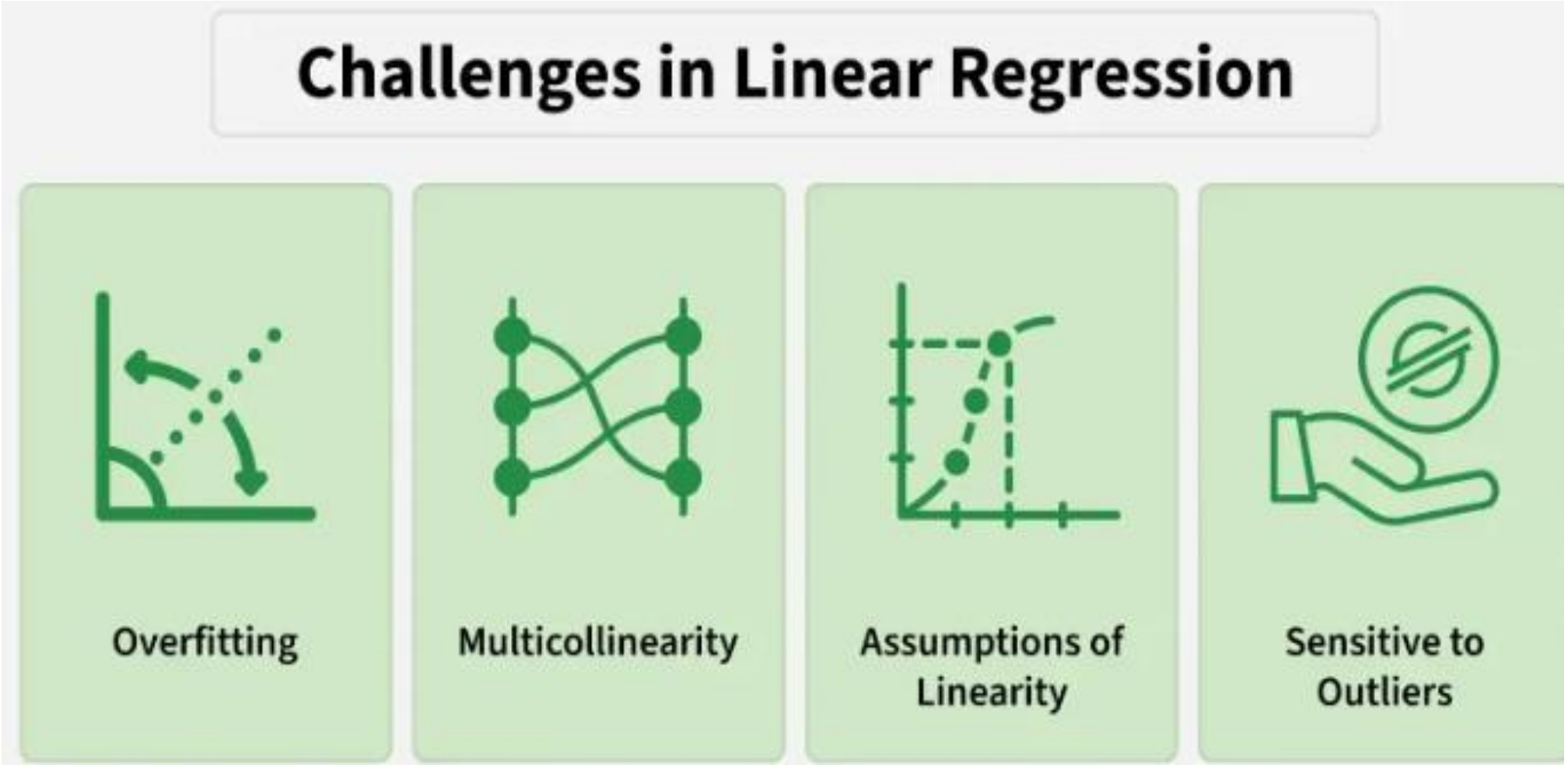
Multiple Linear Regression



Uses two or more independent variables to predict the dependent variable.

UNIVERSITY Of KIGALI

UNIVERSITY Of KIGALI


Challenges in Linear Regression
Overfitting
Multicollinearity
Assumptions of Linearity
Sensitive to Outliers

For example

we want to predict a student's exam score based on how many hours they studied. We observe that as students study more hours, their scores go up. In the example of predicting exam scores based on hours studied. Here

- **Independent variable (input):** Hours studied because it's the factor we control or observe.
- **Dependent variable (output):** Exam score because it depends on hobw many hours were studied

We use the independent variable to predict the dependent variable

Best Fit Line in Linear Regression

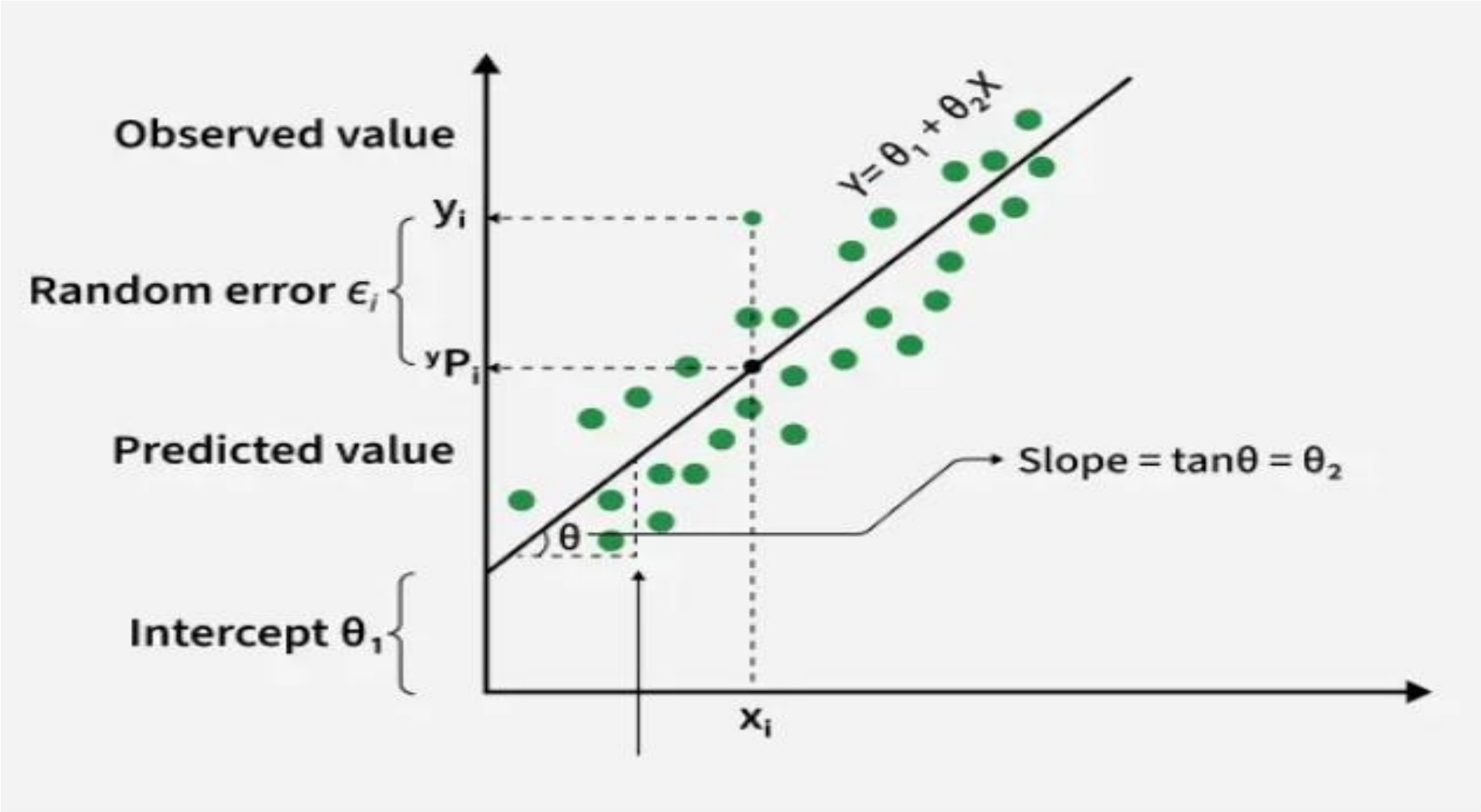
In linear regression, the best-fit line is the straight line that most accurately represents the relationship between the independent variable (input) and the dependent variable (output).

It is the line that minimizes the difference between the actual data points and the predicted values from the model.

1. Goal of the Best-Fit Line

The goal of linear regression is to find a straight line that minimizes the error (the difference) between the observed data points and the predicted values.

This line helps us predict the dependent variable for new, unseen data.


Observed value
y_i
Random error ϵ_i
^yP_i
Predicted value
Intercept θ_1
$Y=\theta_1+\theta_2X$
Slope = $\tan\theta = \theta_2$
θ
x_i

- Here Y is called a dependent or target variable and X is called an independent variable also known as the predictor of Y.
- $\theta 1 \vartheta 1$ represents the intercept, which is the value of Y when X = 0
- $\theta 2 \vartheta 2$ represents the slope, which shows how much Y changes for a unit change in X

2. Equation of the Best-Fit Line

- For simple linear regression (with one independent variable), the best-fit line is represented by the equation
- *y=mx+bSum of squared errors (SSE)* = y=mx+by=*mx+b*

- **Where:**
- **y** is the predicted value (dependent variable)
- **x** is the input (independent variable)
- **m** is the slope of the line (how much y changes when x changes)
- **b** is the intercept (the value of y when x = 0)

UNIVERSITY Of KIGALI

The best-fit line will be the one that optimizes the values of m (slope) and b (intercept) so that the predicted y values are as close as possible to the actual data points.

3. Minimizing the Error: The Least Squares Method

- To find the best-fit line, we use a method called Least Squares. The idea behind this method is to minimize the sum of squared differences between the actual values (data points) and the predicted values from the line. These differences are called residuals.
- The formula for residuals is:

$$Residual = y_i - \hat{y}_i$$

Where:

- y_i is the actual observed value
- $\hat{y}_i$ is the predicted value from the line for that x_i

The least squares method minimizes the sum of the squared residuals:

$$\Sigma(y_i - \hat{y}_i)^2$$

This method ensures that the line best represents the data where the sum of the squared differences between the predicted values and actual values is as small as possible.

4. Interpretation of the Best-Fit Line

Slope (m): The slope of the best-fit line indicates how much the dependent variable (y) changes with each unit change in the independent variable (x). For example if the slope is 5, it means that for every 1-unit increase in x, the value of y increases by 5 units.

Intercept (b): The intercept represents the predicted value of y when x = 0. It's the point where the line crosses the y-axis.

UNIVERSITY Of KIGALI

In linear regression some hypothesis are made to ensure reliability of the model's results.

Limitations:

- **Assumes Linearity:** The method assumes the relationship between the variables is linear. If the relationship is non-linear, linear regression might not work well.
- **Sensitivity to Outliers:** Outliers can significantly affect the slope and intercept, skewing the best-fit line

Hypothesis function in Linear Regression

- In linear regression, the hypothesis function is the equation used to make predictions about the dependent variable based on the independent variables. It represents the relationship between the input features and the target output.
- For a simple case with one independent variable, the hypothesis function is:

$$h(x) = \beta_0 + \beta_1 x$$

Where:

- h(x)$h(x)$ or (y^)$(y\hat{})$ is the predicted value of the dependent variable (y).
- xxis the independent variable.
- $\beta_0 \beta_0$ is the intercept, representing the value of y when x is 0.
- $\beta_1 \beta_1$ is the slope, indicating how much y changes for each unit change in x.

Simple Linear Regression in Python

- Simple linear regression models the relationship between a dependent variable and a single independent variable.
- In this article, we will explore simple linear regression and it's implementation in Python using libraries such as NumPy, Pandas, and scikit-learn.

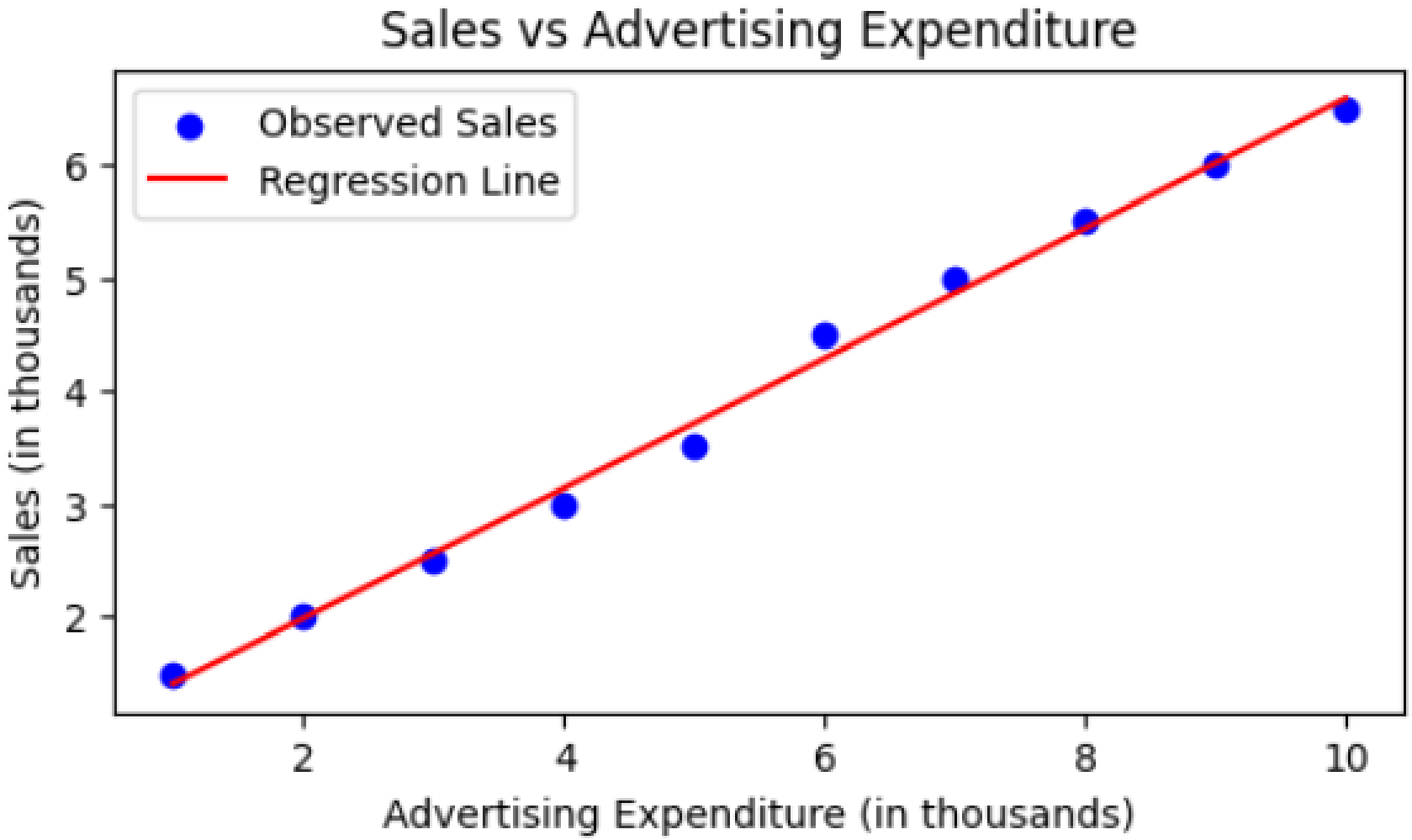
Understanding Simple Linear Regression

Simple Linear Regression aims to describe how one variable i.e the dependent variable changes in relation with reference to the independent variable.

For example consider a scenario where a company wants to predict sales based on advertising expenditure.

By using simple linear regression the company can determine if an increase in advertising leads to higher sales or not.

The below graph explains the relationship between advertising expenditure and sales using simple linear regression:



The relationship between the dependent and independent variables is represented by the simple linear equation:

$$y = mx + b$$

Here:

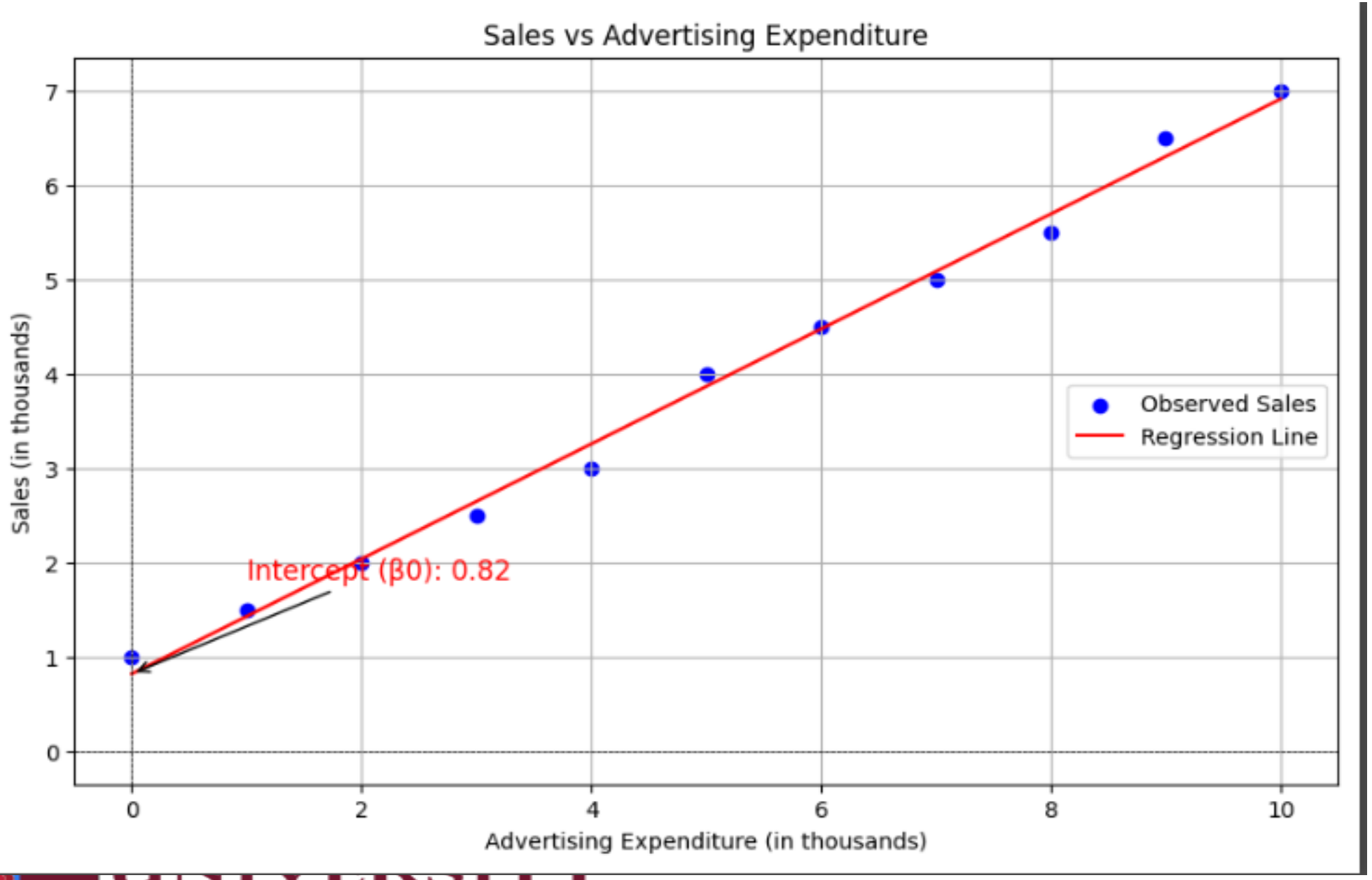
- y is the predicted value (dependent variable).
- m is the slope of the line
- x is the independent variable.
- b is the y-intercept (the value of y when x is 0).

- To better understand this relationship we can express it in a more statistical context using the linear regression formula:
- $Y=\beta 0+\beta 1x$ $Y=\beta 0+\beta 1x$
- In simple linear regression the parameters $\beta 0$ and $\beta 1$ $\beta 0$ and $\beta 1$ play crucial roles in defining the relationship between the independent variable X and the dependent variable Y in the regression equation.

Intercept β0*b*0:

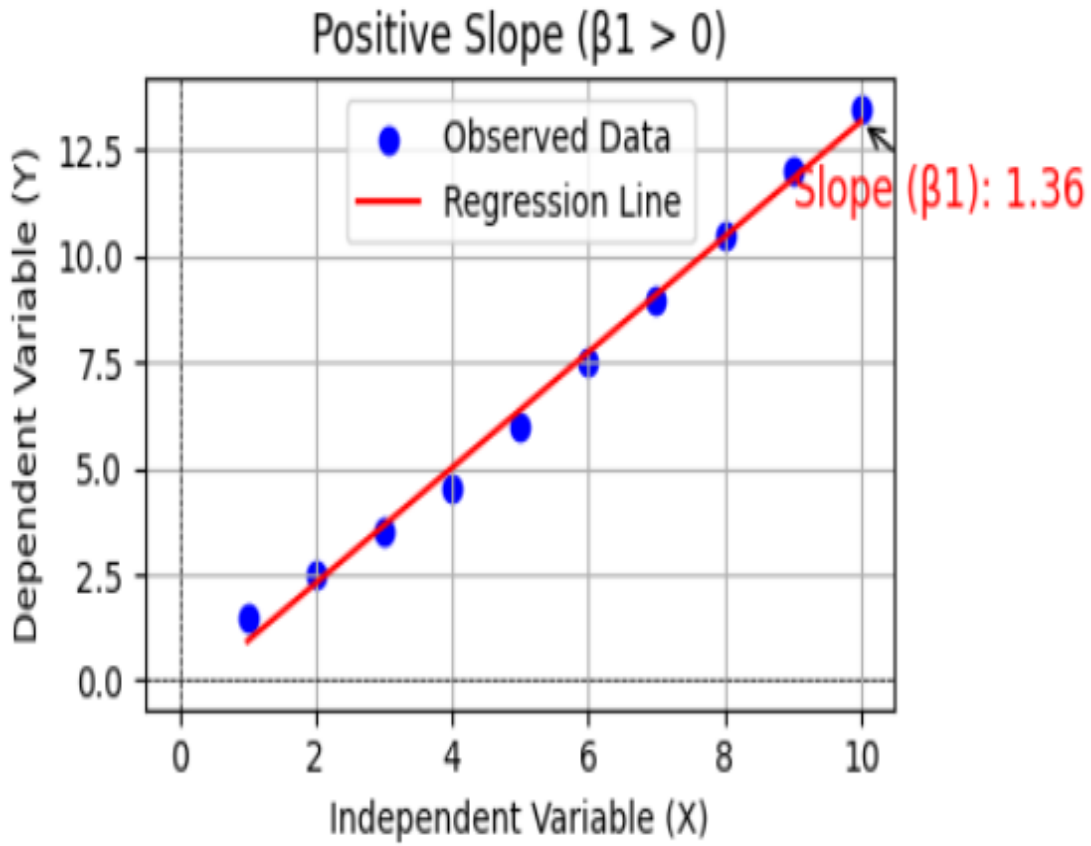
- The intercept$\beta 0b0$ represents the predicted value of the dependent variable Y when the independent variable X is zero. In other words it is the point where the regression line crosses the y-axis.
- It provides a baseline value for Y and helps us understand the expected outcome when there is no influence from X.
- For example if you were predicting sales based on advertising expenditure it would indicate the estimated sales when no money is spent on advertising

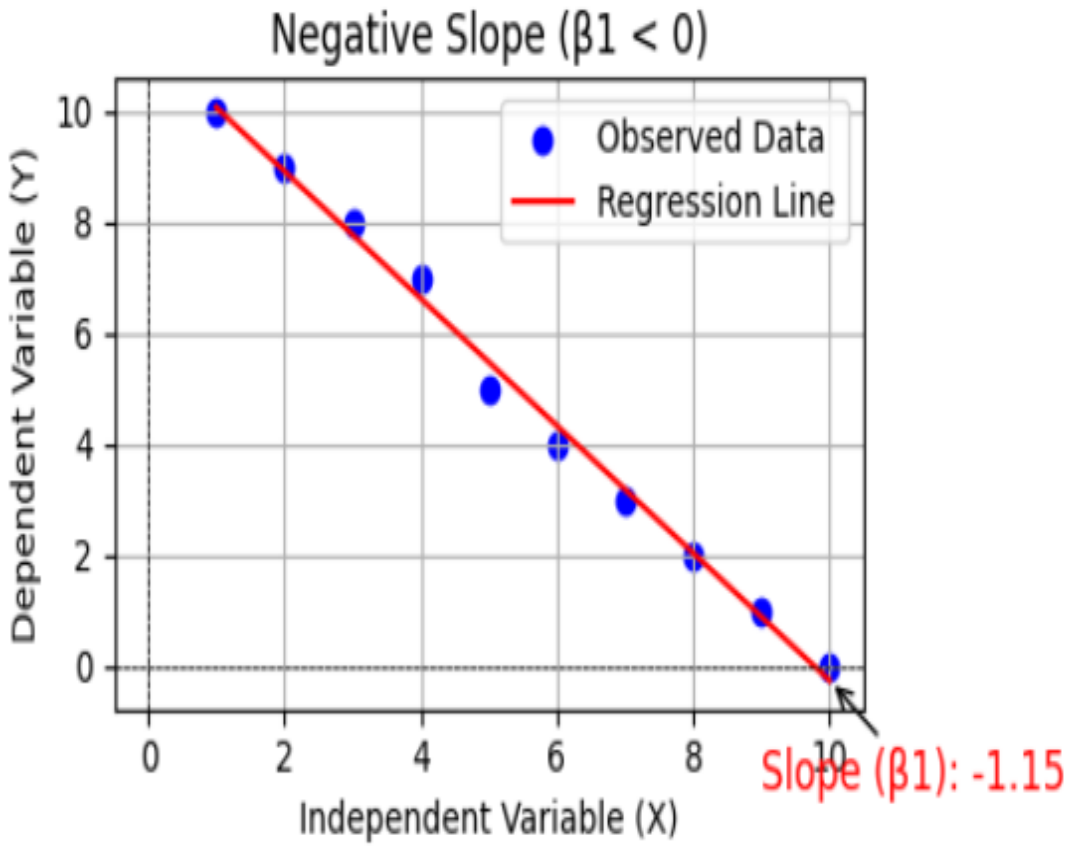
UNIVERSITY Of KIGALI


Sales vs Advertising Expenditure
Sales (in thousands)
Advertising Expenditure (in thousands)
Observed Sales
Regression Line
Intercept (β0): 0.82
0
1
2
3
4
5
6
7
0
2
4
6
8
10

Slope β1β1:

- A positive β1β1 value suggests that as X increases, Y also increases, indicating a direct relationship.
- Conversely, a negative β1β1 value indicates that an increase in X leads to a decrease in Y, indicating an inverse relationship.
- For instance, if β1=2β1=2, this would mean that for every additional unit spent on advertising, sales are expected to increase by 2 units


Positive Slope (β1 > 0)
Observed Data
Regression Line
Slope (β1): 1.36
Dependent Variable (Y)
Independent Variable (X)
0.0
2.5
5.0
7.5
10.0
12.5
0
2
4
6
8
10


Negative Slope (β1 < 0)
Observed Data
Regression Line
Slope (β1): -1.15
Dependent Variable (Y)
Independent Variable (X)
0
2
4
6
8
10
0
2
4
6
8
10

Implementing Simple Linear Regression in Python

Step 1: Import Required Libraries

- import numpy as np
- import pandas as pd
- import matplotlib.pyplot as plt
- from sklearn.model_selection import train_test_split
- from sklearn.linear_model import LinearRegression
- from sklearn.metrics import mean_squared_error, r2_score

numpy (imported as np)

- A fundamental package for numerical computing in Python.
- Provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently.
- Used often for data manipulation and mathematical operations

pandas (imported as pd)

- A powerful library for data manipulation and analysis.
- Provides data structures like DataFrames (2D labeled data) and Series (1D labeled data) that make data handling easy and intuitive.
- Used for loading, cleaning, filtering, and processing datasets, especially tabular data

matplotlib.pyplot (imported as plt)

- A plotting library used to create static, interactive, and animated visualizations in Python.
- pyplot provides a MATLAB-like interface for plotting graphs, histograms, scatter plots, and more.
- Used to visualize data and the results of machine learning models.

sklearn.model_selection.train_test_split

- A utility function from scikit-learn that splits datasets into random train and test subsets.
- Essential for evaluating machine learning models by training on one portion of data and testing on another to prevent overfitting.

sklearn.linear_model.LinearRegression

- The linear regression model implementation in scikit-learn.
- Used to fit a linear model to the data by finding the best-fit line that minimizes the residual sum of squares between observed and predicted values.

sklearn.metrics.mean_squared_error and sklearn.metrics.r2_score

- Functions to evaluate the performance of regression models.
- mean_squared_error calculates the average squared difference between actual and predicted values; lower values indicate better performance.
- r2_score (coefficient of determination) measures how well the regression predictions approximate the real data points; ranges from 0 to 1, with higher values indicating better fit.

Step Two: Load and Prepare the Data

- In this step, you load your dataset and select the features (independent variables) and target (dependent variable) for your regression model

General Syntax:

```
# Load data (example using a CSV file)
data = pd.read_csv('data.csv')
# Select the independent variable(s) - must be 2D array for scikit-learn
X = data[['feature_column']]  # Use double brackets for DataFrame
# Select the dependent variable (target) - 1D array or Series
y = data['target_column']
```

Explanation:

- pd.read_csv('data.csv'): Loads your dataset from a CSV file into a pandas DataFrame. You can also load data from other sources like Excel, databases, or create it manually.
- X: This is your input feature(s). For simple linear regression, it's usually a single column but still passed as a 2D array (hence the double brackets).
- y: This is the output or target variable that your model will learn to predict.

Step Three: Split the Dataset into Training and Testing Sets

- This step divides your data into two parts: one for training the model and the other for evaluating its performance. This helps to assess how well your model generalizes to unseen data.

General Syntax:

```
from sklearn.model_selection import train_test_split

# Split data into training and testing sets
# test_size defines the proportion of data used for testing (e.g., 0.2 means 20%)
# random_state ensures reproducibility by fixing the random seed
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0
```

Explanation:

train_test_split randomly splits the data into training and testing subsets.

X_train and y_train are used to train the model.

X_test and y_test are used to evaluate the model's performance after training.

test_size=0.2 means 20% of the data is reserved for testing; you can adjust this ratio as needed.

random_state=0 ensures that the split is the same every time you run the code, which is useful for reproducibility.

Step Four: Create and Train the Model

In this step, you create an instance of the linear regression model and train (fit) it on the training data.

General Syntax:

```
from sklearn.linear_model import LinearRegression
# Create the Linear Regression model
model = LinearRegression()

#Train the model using the training data
model.fit(X_train, y_train)
```

Explanation:

- LinearRegression() creates a new linear regression model object.
- model.fit(X_train, y_train) fits the model to the training data by finding the best-fitting line that minimizes the error between predicted and actual values

Step Five: Make Predictions

- After training the model, use it to predict the target values for the test (unseen) data.

General Syntax:

Use the trained model to predict target values for the test set

y_pred = model.predict(X_test)

Explanation:

model.predict(X_test) uses the learned relationship from training to estimate the target values corresponding to the features in the test set.

The output y_pred is an array of predicted values for comparison against the actual values y_test.

Example:

```
# Predict salaries based on years of experience in the test set
y_pred = model.predict(X_test)
print("Predicted values:", y_pred)
print("Actual values:", y_test.values)
```

Step Six: Evaluate the Model

- In this step, you assess how well your linear regression model performs by comparing the predicted values with the actual target values from the test set.

Common Metrics:

- Mean Squared Error (MSE): Measures the average squared difference between predicted and actual values. Lower values indicate better fit.
- R^2 Score (Coefficient of Determination): Indicates the proportion of variance in the dependent variable explained by the independent variable(s). Values closer to 1 mean better fi

General Syntax:

```
from sklearn.metrics import mean_squared_error, r2_score
# Calculate Mean Squared Error
mse = mean_squared_error(y_test, y_pred)
# Calculate R² Score
r2 = r2_score(y_test, y_pred)
print(f"Mean Squared Error: {mse}")
print(f"R² Score: {r2}")
```

Explanation:

- mean_squared_error(y_test, y_pred) computes the average squared errors between true and predicted values.
- r2_score(y_test, y_pred) gives a measure of goodness-of-fit; 1 means perfect fit, 0 means the model does no better than the mean.

Example:

- mse = mean_squared_error(y_test, y_pred)
- r2 = r2_score(y_test, y_pred)

- print(f"Mean Squared Error: {mse:.2f}")
- print(f"R^2 Score: {r2:.2f}")

Mean Squared Error (MSE)

- Determining whether a model is "acceptable" based on Mean Squared Error (MSE) and R^2 score depends on the context and specific problem, but here are general guidelines:
- Lower MSE values indicate better model performance because the predictions are closer to actual values.
- However, MSE is in the squared units of the target variable, so its magnitude depends on your data scale.
- You should compare MSE against baseline models

(e.g., predicting the mean) or previous models to judge acceptability.

UNIVERSITY Of KIGALI

When to say a model is acceptable:

$R^2 \geq 0.7$: Often considered a good fit in many practical scenarios.

$0.4 \leq R^2 < 0.7$: Moderate fit, may be acceptable depending on the domain.

$R^2 < 0.4$: Poor fit, usually indicates the model is not capturing the relationship well.

MSE should be as low as possible, but what is "low" depends on the problem scale

Important:

Always consider domain knowledge, business requirements, and baseline models.

Check residual plots and other diagnostics beyond just these metrics.

Sometimes a lower R^2 might be acceptable if the problem is inherently noisy or complex.

Example Scenario:

Suppose you trained a simple linear regression model to predict salaries based on years of experience. After evaluating on the test set, you got:

- Mean Squared Error: 25000000.0
- R^2 Score: 0.85

Interpretation:

MSE = 25,000,000

This means on average, the squared difference between the predicted salary and actual salary is 25 million (in whatever currency squared). Since salary values can be in tens or hundreds of thousands, this may be reasonable. To better understand, you can take the square root to get RMSE (Root Mean Squared Error):

[RMSE = \sqrt{25,000,000} = 5000]

This means predictions are on average about $5,000 off, which might be acceptable depending on the salary range.

$R^2 = 0.85$

This means 85% of the variance in salaries is explained by years of experience in this model, which is quite good. It indicates a strong predictive relationship.

Another Example:

Mean Squared Error: 90000000.0

R^2 Score: 0.3

MSE = 90,000,000

RMSE = 9487. This is a higher average error, almost $9,500 off on predictions.

R^2 = 0.3

Only 30% of the variance is explained by the model, which is relatively low. This suggests the model is not capturing the relationship well and might not be acceptable for reliable predictions.

Summary Table for Reference:

Metric	Result Example	Interpretation
MSE	25,000,000	RMSE ~$5,000 — moderate error
R^2	0.85	Good fit; model explains 85% variance
MSE	90,000,000	RMSE ~$9,500 — higher error
R^2	0.3	Poor fit; model explains only 30% variance

Step Six: Evaluate the Model

- In this step, you assess the performance of your trained linear regression model by comparing its predictions on the test data with the actual target values. This helps you understand how accurately your model can make predictions on unseen data.

Key Evaluation Metrics:

- Mean Squared Error (MSE): Measures the average squared difference between predicted and actual values. Lower is better.
- R^2 Score (Coefficient of Determination): Indicates how well the model explains the variance in the target variable. Values closer to 1 mean a better fit.

R^2 Score (Coefficient of Determination)

R^2 ranges from 0 to 1 (sometimes can be negative if model is very poor).

R^2 close to 1 means the model explains most of the variability in the target variable generally considered good.

R^2 near 0 means the model explains very little; predictions are no better than simply using the mean of the target.

Negative R^2 means the model fits worse than a horizontal line (mean prediction).

Step 8: Visualize Actual vs Predicted Results

```
plt.scatter(X_test, y_test, color='blue', label='Actual Salary')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Predicted Salary')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.title('Simple Linear Regression: Salary vs Experience')
plt.legend()
plt.show()
```